Real-world (build and compare the supervisor version) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

A minimal supervisor graph needs:

- A shared state shape holding the task, the research result, the write result, and a routing history log.
- A `supervisor` node that looks at what's been done so far and decides, via `Command`, which specialist runs next (or that the task is done).
- Two specialist nodes (`research_node`, `write_node`) wrapping your existing `research()` / `write()` functions, each returning to the supervisor when done.

Look specifically at:

- **`Command(update=..., goto=...)`** — the supervisor node returns one of these instead of a plain dict; `update` carries the state changes, `goto` names the next node (or `END`).
- **The supervisor's own decision cost** — this is a real LLM call (or, for a first version, simple logic checking what's already in state), and it needs to be measured the same way `research` / `write` are, since it's the extra cost this pattern is being blamed or credited for.
- **Running the identical 3 test inputs through both the sequential pipeline and this supervisor graph** — same topics, same model, so the comparison is fair.

Reuse `sequential_measure`'s `StageResult`-style measurement inside each node, so you get directly comparable numbers.

Sketch the state shape and the three nodes (supervisor, research, write), then go one step further: think about the fairest way to actually credit or blame the supervisor pattern. A supervisor using plain `if` checks — "if no research_text yet, go to research" — costs nothing. That's a fair simplification for a fixed two-step task, but it also quietly hides the exact cost this whole exercise is supposed to be

measuring: a *real* routing decision, made by an LLM, has its own token cost and its own latency, and a supervisor built entirely from `if` checks will never show you that.

The real design question isn't just "build a graph with a router node" — it's "does my router's own decision cost get measured and reported like every other stage, or is it invisible in my numbers because I built it out of free Python logic instead of a real LLM call?"

The extra pieces needed:

- An LLM-based supervisor variant — one that actually calls the model to decide `goto="research_node"` vs `goto="write_node"` vs `goto=END`, with that call's own tokens and seconds recorded into the state the same way `research_node` / `write_node` do.
- A side-by-side run of both supervisor variants (`if`-based and LLM-based) against the sequential baseline, so you can see the true spread: sequential's cost, the cheapest possible supervisor (free routing logic), and a realistic supervisor (paid routing logic).
- A written conclusion naming which of the three actually won on this task, and by how much — not just which one you expected to win.

Sketch this hardened version yourself before checking Hint 2.

Hint 2 — The plan, and almost the whole thing

```
# architecture_comparison_practice.py — Real-world section
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command

class SupervisorState(TypedDict):
    task: str
    research_text: str
    summary: str
    routing_log: list
    total_tokens: int
    total_seconds: float

def supervisor(state: SupervisorState) -> Command:
    if not state.get("research_text"):
        log = state["routing_log"] + ["research"]
        return Command(update={"routing_log": log}, goto="research_node")
    if not state.get("summary"):
        log = state["routing_log"] + ["write"]
        return Command(update={"routing_log": log}, goto="write_node")
    return Command(goto=END)

def research_node(state: SupervisorState) -> Command:
```

```
    result = research(state["task"])  # from sequential_measure
    return Command(
        update={
            "research_text": result.text,
            "total_tokens": state["total_tokens"] + result.tokens,
            "total_seconds": state["total_seconds"] + result.seconds,
        },
        goto="supervisor",
    )

def write_node(state: SupervisorState) -> Command:
    result = write(state["research_text"])  # from sequential_measure
    return Command(
        update={
            "summary": result.text,
            "total_tokens": state["total_tokens"] + result.tokens,
            "total_seconds": state["total_seconds"] + result.seconds,
        },
        goto="supervisor",
    )
```

Notice the supervisor itself, in this simplest version, uses plain `if` checks rather than its own LLM call — that's a fair simplification for a fixed two-step task, and it's worth trying the LLM-based version too, below, to measure the routing call's own real cost.

Wire the nodes into a graph and run the same 3 topics you used in `sequential_measure`:

```
# architecture_comparison_practice.py — Real-world section
builder = StateGraph(SupervisorState)
builder.add_node("supervisor", supervisor)
builder.add_node("research_node", research_node)
builder.add_node("write_node", write_node)
builder.add_edge(START, "supervisor")
graph = builder.compile()
```

Print `routing_log`, `total_tokens`, and `total_seconds` for each of the 3 topics, next to the same numbers from `sequential_measure`, then go one step further, into the LLM-based supervisor variant below:

```
LLM-based supervisor node:
    build a short prompt describing the current state
    (has research happened? has write happened?)
    ask the model to answer with exactly one word: RESEARCH, WRITE, or DONE
    time this call and read its tokens, same as any other stage
    add those tokens/seconds into total_tokens/total_seconds, same
```

```
    field, same bucket
    map the model's word to goto="research_node" / "write_node" / END
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# architecture_comparison_practice.py — Real-world section
import time

def supervisor_llm(state: SupervisorState) -> Command:
    start = time.perf_counter()
    prompt = (
        f"Task: {state['task']}\n"
        f"Research done: {bool(state.get('research_text'))}\n"
        f"Write done: {bool(state.get('summary'))}\n"
        "Reply with exactly one word: RESEARCH, WRITE, or DONE."
    )
    response = model.invoke(prompt)
    elapsed = time.perf_counter() - start
    decision = response.content.strip().upper()
    call_tokens = response.usage_metadata["total_tokens"]

    updates = {
        "total_tokens": state["total_tokens"] + call_tokens,
        "total_seconds": state["total_seconds"] + elapsed,
    }

    # your turn: map `decision` to the right goto target and routing_log entry,
    # the same way the if-based supervisor() above does
    ...
```

Fill in the mapping yourself, run both supervisor variants against the same 3 topics and against `sequential_measure`'s numbers, then compare your written conclusion against the Solution.

Full solution: Show me the solution